



Database (SQL Server) Table Partitioning

What is Table Partitioning

- Technique in SQL Server that allows you to split a large table into smaller, more manageable pieces (partitions) based on the values in a specific column
- Each partition behaves like a separate table internally
- Partitions are invisible to the user: Treated as one entity when queried.

Table Partitioning

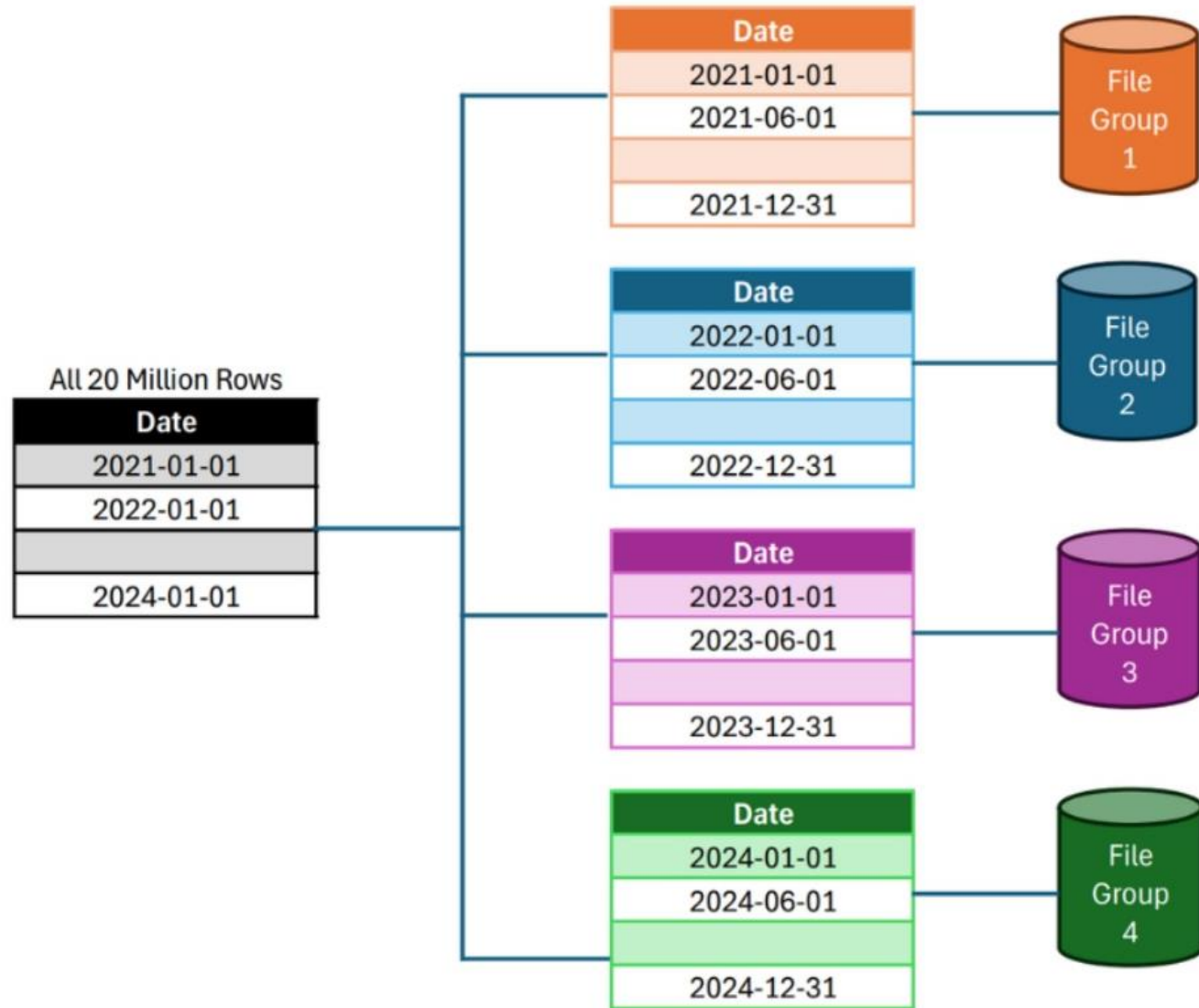
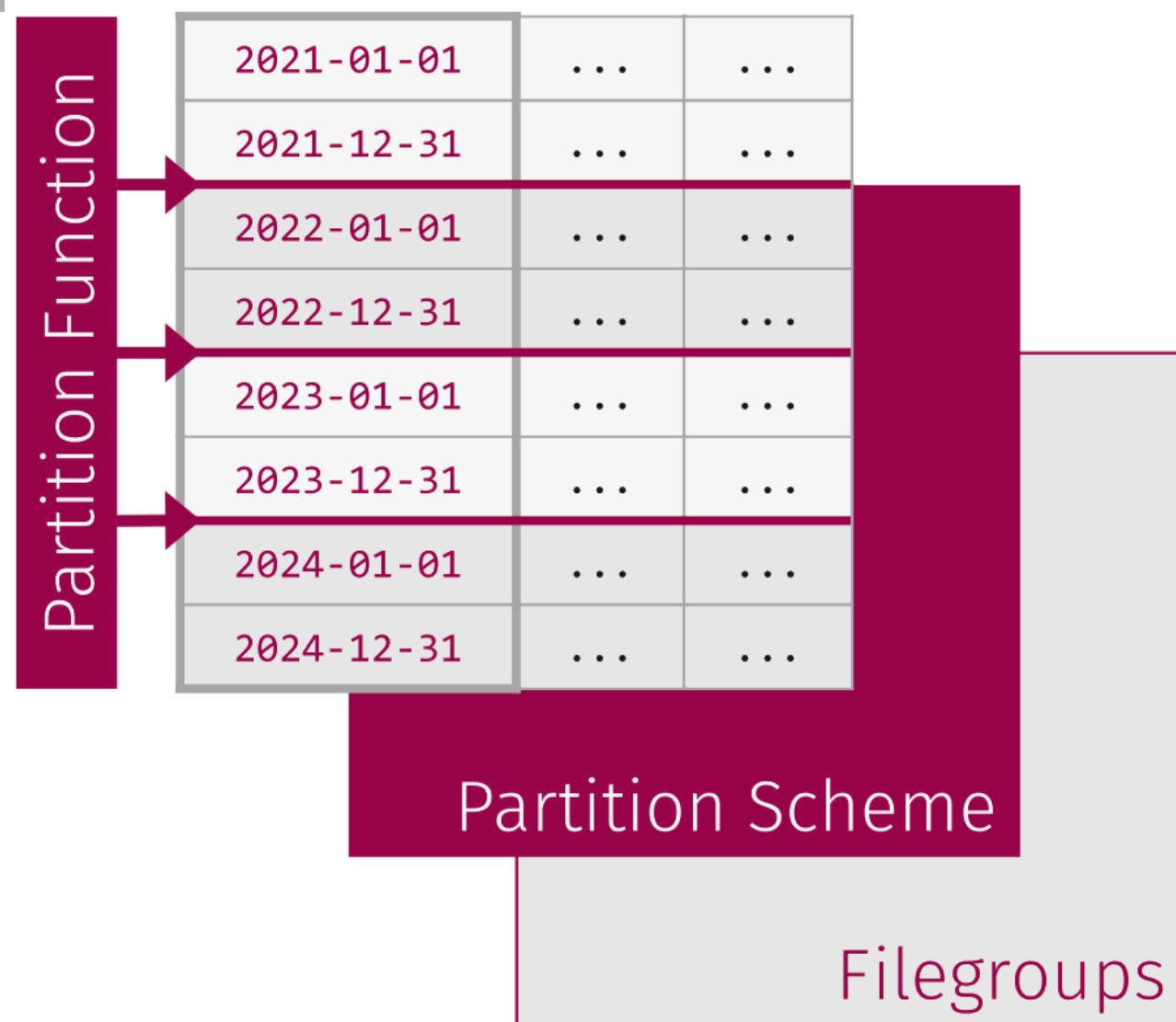
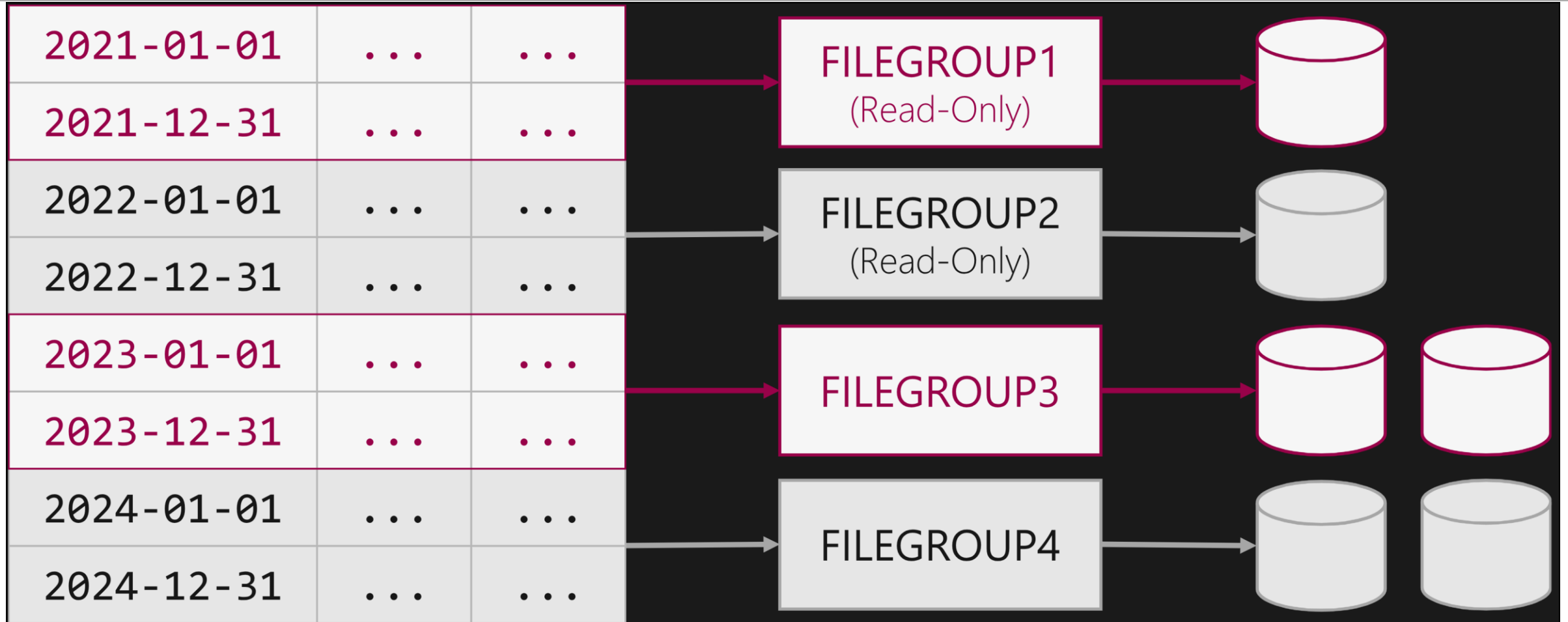


Table Partitioning

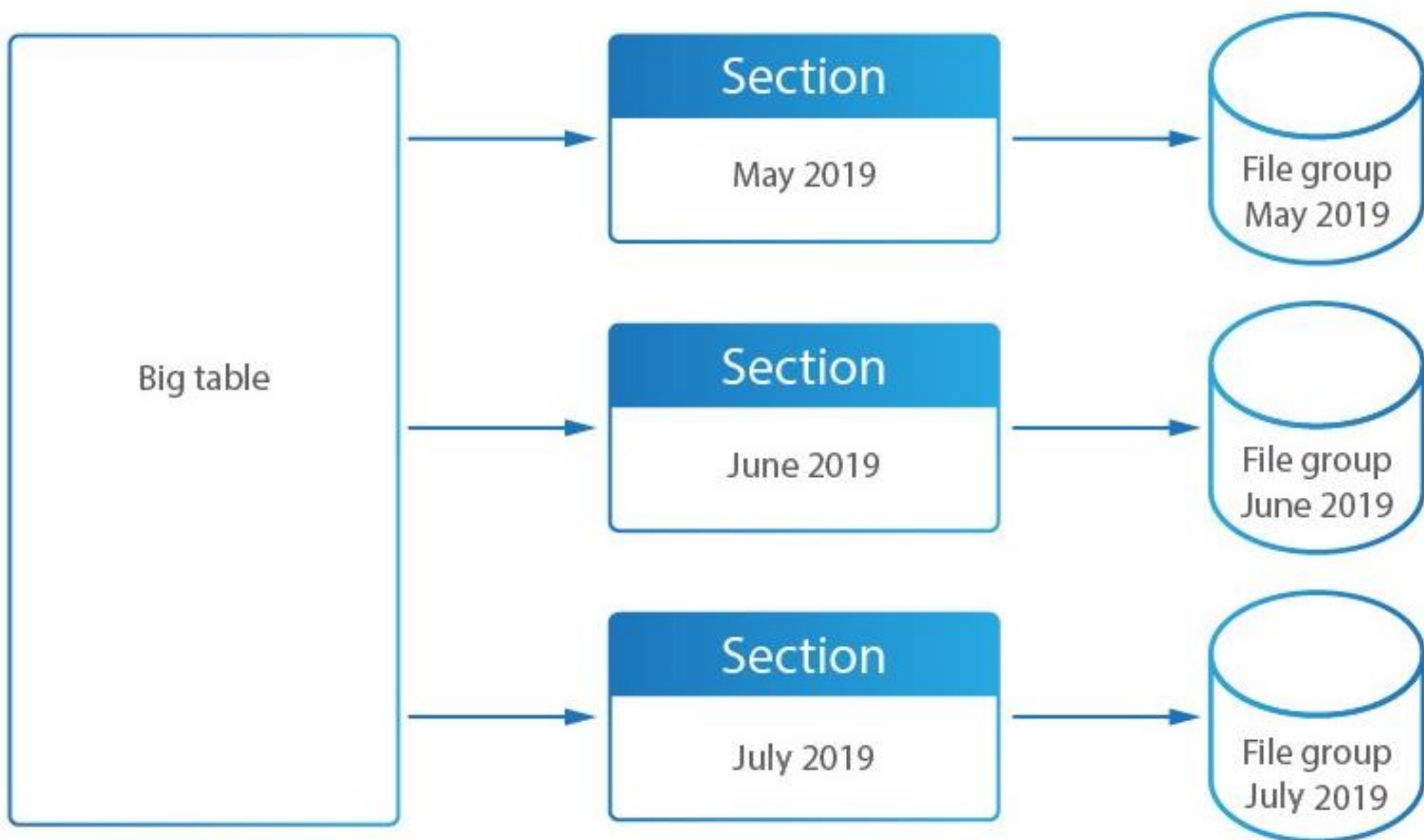


Partitioning Scheme



Partitioning function

Partitioning schema



2. Partition Function

- Defines how the rows will be mapped to the partitions
- Based on the partitioning column

```
CREATE PARTITION FUNCTION OrderDatePF (datetime)  
AS RANGE RIGHT FOR VALUES  
( '2018/01/01' , '2019/01/01' , '2020/01/01' );
```

RANGE RIGHT means that each boundary value belongs to the partition to its right.

Function – OrderDatePF

Partition1	Partition2	Partition3	Partition4
OrderDate < 2018/01/01	OrderDate >= 2018/01/01 AND < 2019/01/01	OrderDate >= 2019/01/01 AND < 2020/01/01	OrderDate >= 2020/01/01

Values: Minimum to Maximum

Range Types – RIGHT or LEFT

- As part of the function, you need to determine if your partition will range right or range left.
- Range right - "begins at" - lower boundary
- Range left - "ends at" - upper boundary

```
CREATE PARTITION FUNCTION OrderDatePF (datetime)
AS RANGE RIGHT FOR VALUES
( '2018/01/01' , '2019/01/01' , '2020/01/01' );
```

Range Type Example – Unpartitioned Table



RANGE RIGHT

```
CREATE PARTITION FUNCTION PF1_Right (int)  
AS RANGE RIGHT FOR VALUES (10);
```

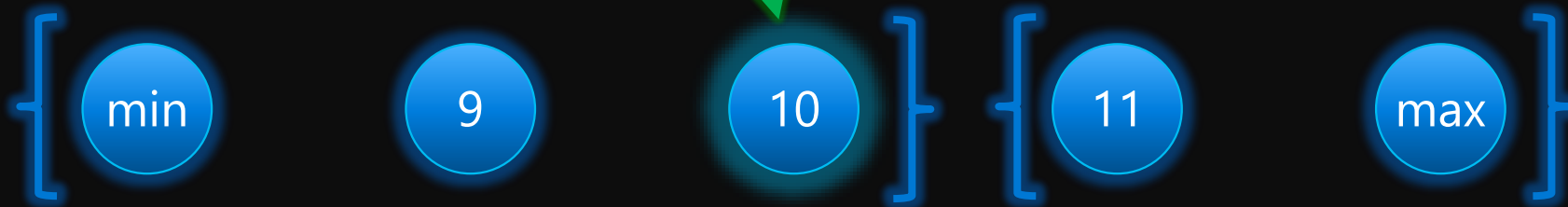


Partition 1

Partition 2

RANGE LEFT

```
CREATE PARTITION FUNCTION PF1_Left (int)  
AS RANGE LEFT FOR VALUES (10);
```



Partition 1

Partition 2



Partitioning Scheme

Partitioning Scheme

A **partitioning scheme** defines **how data is distributed across partitions** based on a **partition function**.

- The partition function, as defined earlier, maps rows to partitions based on the values in a specified column (often a date or ID).
- Then partitioning scheme maps those partitions to filegroups.

Benefits of Partitioning:

- **Improved Query Performance:** Queries can target specific partitions.
- **Efficient Maintenance:** You can switch, truncate, or rebuild partitions individually.
- **Better Storage Management:** Distribute data across multiple filegroups/disks.

Partitioning Scheme to Map Each Partition to different filegroup

```
CREATE PARTITION SCHEME YearPS  
AS PARTITION OrderDatePF  
TO (FG1, FG2, FG3, FG4);
```

```
CREATE TABLE Orders  
    (OrderId INT, OrderDate DATETIME, Col3,etc...)  
ON YearPS (OrderDate);
```

Partitioning Scheme: Example

Imagine a table storing sales data for multiple years. You can partition it by year:

```
CREATE PARTITION FUNCTION pfYear(INT)  
AS  
RANGE LEFT FOR VALUES (2019,2020,2021,2022) ;
```

The above maps each year to a different filegroup and creates a table using this scheme. Queries for a specific year will only scan the relevant partition.

Partitioning Scheme

